Unit 2

Key Features of Data Warehouse

The key features of a data warehouse are discussed below:

- **Subject Oriented** - A data warehouse is subject oriented because it provides information around a subject rather than the organization's ongoing operations. These subjects can be product, customers, suppliers, sales, revenue, etc. A data warehouse does not focus on the ongoing operations; rather it focuses on modeling and analysis of data for decision making.
- **Integrated** - A data warehouse is constructed by integrating data from heterogeneous sources such as relational databases, flat files, etc. This integration enhances the effective analysis of data.
- **Time Variant** - The data collected in a data warehouse is identified with a particular time period. The data in a data warehouse provides information from the historical point of view.
- **Non-volatile** - Non-volatile means the previous data is not erased when new data is added to it. A data warehouse is kept separate from the operational database and therefore frequent changes in operational database are not reflected in the data warehouse.

DBMS vs. Data Warehouse

The major task of database systems is to perform on-line transaction and query processing. These systems are called on-line transaction processing (OLTP) systems. They cover most of the day-to-day operations of an organization, such as purchasing, inventory, manufacturing, banking, payroll, registration, and accounting. Data warehouse systems, on the other hand, serve users or knowledge workers in the role of data analysis and decision making. Such systems can organize and present data in various formats in order to accommodate the diverse needs of the different users. These systems are known as on-line analytical processing (OLAP) systems. The major distinguishing features between OLTP and OLAP are summarized as follows:

→ **Users and System Orientation**: An OLTP system is *customer-oriented* and is used for transaction and query processing by clerks, clients, and information technology professionals. An OLAP system is *market-oriented* and is used for data analysis by knowledge workers, including managers, executives, and analysts.

→ **Data Contents:** An OLTP system manages current data that, typically, are too detailed to be easily used for decision making. An OLAP system manages large

amounts of historical data, provides facilities for summarization and aggregation, and stores and manages information at different levels of granularity. These features make the data easier to use in informed decision making.

→ **Database Design**: An OLTP system usually adopts an entity-relationship (ER) data model and an application-oriented database design. An OLAP system typically adopts either a *star* or *snowflake* model (to be discussed in Section 3.2.2) and a subject oriented database design.

→ **View:** An OLTP system focuses mainly on the current data within an enterprise or department, without referring to historical data or data in different organizations. In contrast, an OLAP system often spans multiple versions of a database schema, due to the evolutionary process of an organization. OLAP systems also deal with information that originates from different organizations, integrating information from many data stores. Because of their huge volume, OLAP data are stored on multiple storage media.

→ **Access Patterns:** The access patterns of an OLTP system consist mainly of short, atomic transactions. Such a system requires concurrency control and recovery mechanisms. However, accesses to OLAP systems are mostly read-only operations (because most data warehouses store historical rather than up-to-date information)

Why Separate Data Warehouse?

Databases store huge amounts of data. Now the major question is *"why not perform on-line analytical processing directly on such databases instead of spending additional time and resources to construct a separate data warehouse?"* A major reason for such a separation is to help promote the *high performance of both systems*.

→ An operational database is designed and tuned from known tasks and workloads, such as indexing and hashing using primary keys, searching for particular records, and optimizing canned queries. On the other hand, data warehouse queries are often complex. They involve the computation of large groups of data at summarized levels, and may require the use of special data organization, access, and implementation methods based on multidimensional views. Processing OLAP queries in operational databases would substantially degrade the performance of operational tasks.

→ Concurrency control and recovery mechanisms, such as locking and logging, are required to ensure the consistency and robustness of transactions in database systems. An OLAP query often needs read-only access of data records for summarization and aggregation. Concurrency control and recovery mechanisms,

if applied for such OLAP operations may jeopardize the execution of concurrent transactions and thus substantially reduce the throughput of an OLTP system.

→ Finally, the separation of operational databases from data warehouses is based on the different structures, contents, and uses of the data in these two systems. Decision support requires historical data, whereas operational databases do not typically maintain historical data. In this context, the data in operational databases, though abundant, is usually far from complete for decision making.

Multidimensional Data Model

Data warehouses and OLAP tools are based on a multidimensional data model. This model views data in the form of a *data cube*.

From Tables and Spreadsheets to Data Cubes

A data cube allows data to be modeled and viewed in multiple dimensions. It is defined by *dimensions* and *facts*. Dimensions are the entities with respect to which an organization wants to keep records. For example, an organization may create a *sales* data warehouse in order to keep records of the store's sales with respect to the dimensions *time*, *item*, *branch*, and *location*. Each dimension may have a table associated with it, called a *dimension table*. This table further describes the dimensions. For example, a dimension table for *item* may contain the attributes *item name, brand*, and *type*.

A multidimensional data model is typically organized around a central theme, like *sales*. This theme is represented by a fact table. Facts are numerical measures. Themes are the quantities by which we want to analyze relationships between dimensions. Examples of facts for a sales data warehouse include *dollars_sold* (sales amount in dollars), *units_sold* (number of units sold), and *amount budgeted*. The fact table contains the names of the *facts*, or measures, as well as keys to each of the related dimension tables.

	location = "Chicago"				location = "New York"				location = "Toronto"				location = "Vancouver"			
	item				item				item				item			
time	home ent.	comp.	phone	sec.	home ent.	comp.	phone	sec.	home ent.	comp.	phone	sec.	home ent.	comp.	phone	sec.
Q1	854	882	89	623	1087	968	38	872	818	746	43	591	605	825	14	400
Q2	943	890	64	698	1130	1024	41	925	894	769	52	682	680	952	31	512
Q3	1032	924	59	789	1034	1048	45	1002	940	795	58	728	812	1023	30	501
Q4	1129	992	63	870	1142	1091	54	984	978	864	59	784	927	1038	38	580

Figure: *Sales data for an organization according to the dimensions time, item, and location. The measure displayed is dollars_sold.*

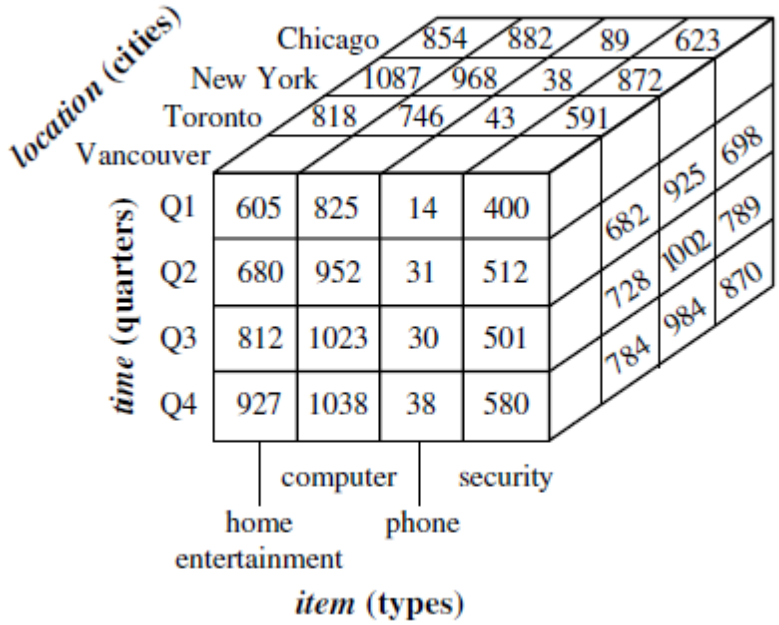


Figure A 3-D data cube representation of the data in above table, according to the dimensions time, item, and location. The measure displayed is dollars_sold (in thousands).

Suppose that we would now like to view our sales data with an additional fourth dimension, such as *supplier*. Viewing things in 4-D becomes tricky. However, we can think of a 4-D cube as being a series of 3-D cubes, as shown in Figure below.

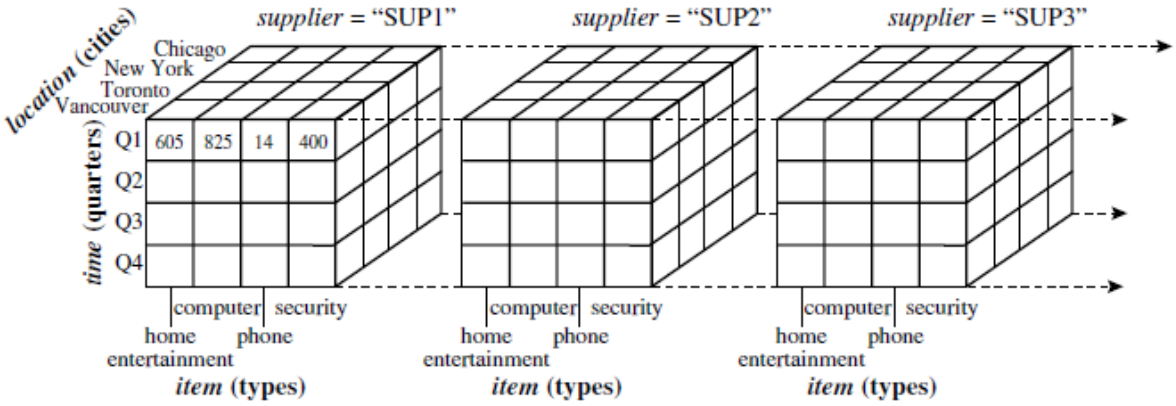


Figure 4-D data cube representation of sales data, according to the dimensions time, item, location, and supplier. The measure displayed is dollars_sold (in thousands)

If we continue in this way, we may display any *n*-dimensional data as a series of (*n*-1) dimensional cubes. The data cube is a metaphor for multidimensional data storage. The actual physical storage of such data may differ from its logical representation. The

important thing to remember is that data cubes are *n*-dimensional and do not confine data to 3-D.

Schemas for Multidimensional Database

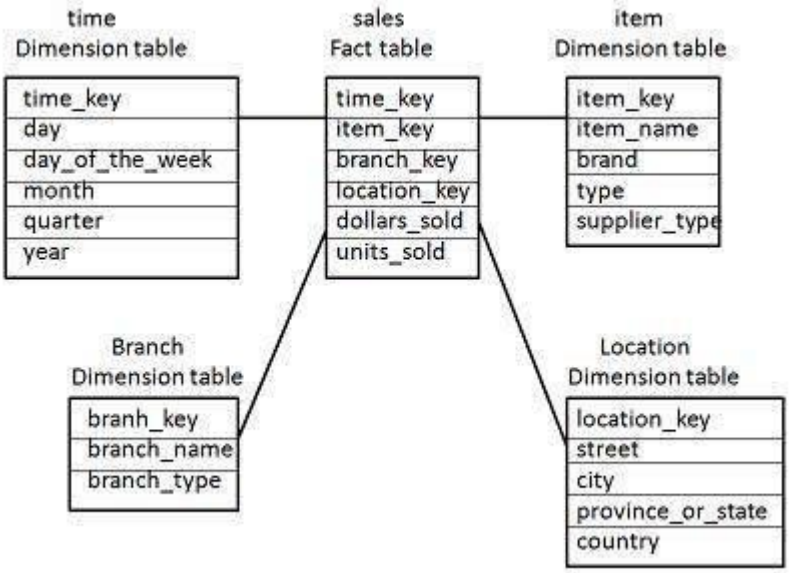
Schema is a logical description of the entire database. It includes the name and description of records of all record types including all associated data-items and aggregates. A data warehouse uses *Star, Snowflake, and Fact Constellation schema*.

Start Schema

It is the data warehouse schema that contains two types of tables: *Fact Table and Dimension Tables.* Fact Table lies at the center point and dimension tables are connected with fact table such that star share is formed.

- → **Fact Tables:** A fact table typically has two types of columns: foreign keys to dimension tables and measures those that contain numeric facts. A fact table can contain fact's data on detail or aggregated level.
- → **Dimension Tables:** Dimension tables usually have a relatively small number of records compared to fact tables, but each record may have a very large number of attributes to describe the fact data.

Each dimension in the star schema has only one dimension table and each table holds a set of attributes. This constraint may cause data redundancy. The following diagram shows the sales data of a company with respect to the four dimensions, namely time, item, branch, and location.

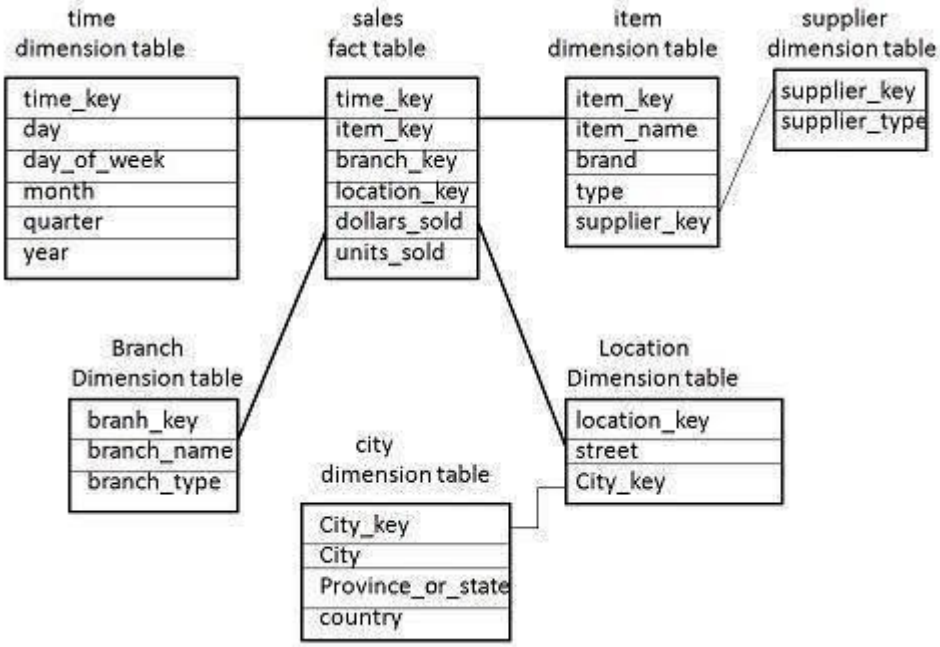


There is a fact table at the center. It contains the keys to each of four dimensions. The fact table also contains the attributes, namely dollars sold and units sold.

Since star schema contains de-normalized dimension tables, it leads to simpler queries due to lesser number of join operations and it also leads to better system performance. On the other hand it is difficult to maintain integrity of data in star schema due to de-normalized tables. It is the wifely used data warehouse schema and is also recommended by oracle

Snowflake Schema

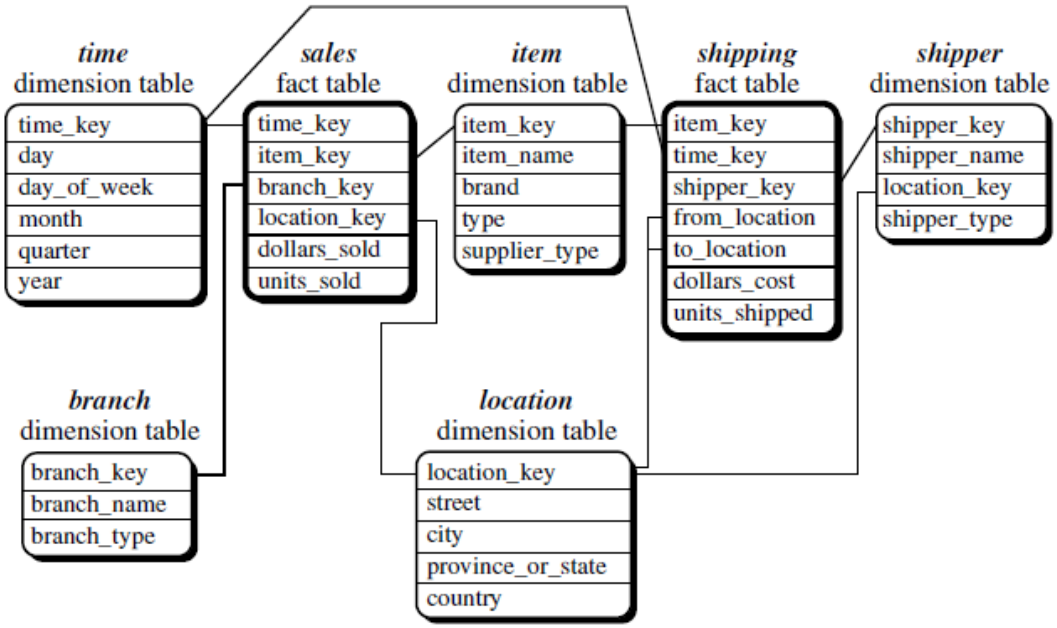
The snowflake schema is a variant of the star schema model, where some dimension tables are *normalized*, thereby further splitting the data into additional tables. The resulting schema graph forms a shape similar to a snowflake. For example, the item dimension table in star schema is normalized and split into two dimension tables, namely item and supplier table.



Due to normalization table is easy to maintain and saves storage space. However, this saving of space is negligible in comparison to the typical magnitude of the fact table. Furthermore, the snowflake structure can reduce the effectiveness of browsing, since more joins will be needed to execute a query. Consequently, the system performance may be adversely impacted. Hence, although the snowflake schema reduces redundancy, it is not as popular as the star schema in data warehouse design.

Fact Constellation Schema

This kind of schema can be viewed as a collection of stars, and hence is called a galaxy schema or a fact constellation. A fact constellation schema allows dimension tables to be shared between fact tables. For example, following schema specifies two fact tables, *sales* and *shipping*. The *sales* table definition is identical to that of the star schema. The *shipping* table has five dimensions, or keys: *item key, time key, shipper key, from location*, and *to location*, and two measures: *dollars cost* and *units shipped*.



Schema Definition

Multidimensional schema is defined using Data Mining Query Language (DMQL). The two primitives, cube definition and dimension definition, can be used for defining the data warehouses and data marts.

Syntax for Cube Definition

define cube < cube_name > [< dimension-list > }: < measure_list>

Syntax for Dimension Definition

define dimension < dimension_name > as (< attribute_or_dimension_list >)

Star Schema Definition

The star schema that we have discussed can be defined using Data Mining Query Language (DMQL) as follows.

define cube sales_star [time, item, branch, location]:
dollars_sold = sum(sales_in_dollars), units_sold = count(*)
define dimension time as (time_key, day, day_of_week, month, quarter, year)
define dimension item as (item_key, item_name, brand, type, supplier_type)
define dimension branch as (branch_key, branch_name, branch_type)
define dimension location as (location_key, street, city, province_or_state, country)

Snowflake Schema Definition

Snowflake schema can be defined using DMQL as follows:

define cube sales snowflake [time, item, branch, location]:
dollars_sold = sum(sales_in_dollars), units_sold = count(*)
define dimension time as (time_key, day, day_of_week, month, quarter, year)
define dimension item as (item_key, item_name, brand, type, supplier (supplier_key, supplier_type))
define dimension branch as (branch_key, branch_name, branch_type)
define dimension location as (location_key, street, city (city_key, city, province_or_state, country)

Fact Constellation Schema Definition

Fact constellation schema can be defined using DMQL as follows:

define cube sales [time, item, branch, location]:
dollars sold = sum(sales in dollars), units sold = count(*)

define dimension time as (time_key, day, day_of_week, month, quarter, year)
define dimension item as (item_key, item_name, brand, type, supplier_type)
define dimension branch as (branch_key, branch_name, branch_type)
define dimension location as (location_key, street, city, province_or_state,country)

define cube shipping [time, item, shipper, from_location, to_location]:
dollars_cost = sum(cost_in_dollars), units_shipped = count(*)
define dimension time as time in cube sales
define dimension item as item in cube sales
define dimension shipper as (shipper_key, shipper_name, location as location in cube sales, shipper_type)

define dimension from_location as location in cube sales
define dimension to_location as location in cube sales

Meta Data

Metadata is simply defined as data about data. The data that is used to represent other data is known as metadata. For example, the index of a book serves as a metadata for the contents in the book. In other words, we can say that metadata is the summarized data that leads us to detailed data. In terms of data warehouse, we can define metadata as follows.

- → Metadata is the road-map to a data warehouse.
- → Metadata in a data warehouse defines the warehouse objects.
- → Metadata acts as a directory. This directory helps the decision support system to locate the contents of a data warehouse.

Metadata can be broadly categorized into three categories:

- → **Business Metadata** - It has the data ownership information, business definition, and changing policies.
- → **Technical Metadata** - It includes database system names, table and column names and sizes, data types and allowed values. Technical metadata also includes structural information such as primary and foreign key attributes and indices.
- → **Operational Metadata** - It includes currency of data and data lineage. Currency of data means whether the data is active, archived, or purged. Lineage of data means the history of data migrated and transformation applied on it.

Data Marts

A data mart is a subject-oriented archive that stores data and uses the retrieved set of information to assist and support the requirements involved within a particular business function or department. Data marts exist within a single organizational data warehouse repository. Data marts improve end-user response time by allowing users to have access to the specific type of data they need to view most often.

A data mart is basically a condensed and more focused version of a data warehouse that reflects the regulations and process specifications of each business unit within an organization. Each data mart is dedicated to a specific business function or region. This subset of data may span across many or all of an enterprise's functional subject areas. It is common for multiple data marts to be used in order to serve the needs of each

individual business unit (different data marts can be used to obtain specific information for various enterprise departments, such as accounting, marketing, sales, etc.).